

University of Tehran

College of Engineering

School of Electrical and Computer Engineering

Computer Architecture Laboratory Report

Group Members:

Faezeh Meysaghi Artin Tavasoli

2025-2026

Contents

1	Simulation Results of the Fetch Stage (IF) and Pipeline Registers	2
2	Simulation Results of the Decode Stage (ID Stage)	2
2.1	Condition Check Module	2
2.2	Register File Initialization Check	3
2.3	Control Unit Performance Verification	3
3	Simulation and Verification of the Execution Stage (EXE Stage)	9
3.1	Arithmetic Logic Unit (ALU) Verification	9
3.2	Status Register Verification and Flag Generation	12
3.3	Branch Address Calculation Verification	13
3.4	Val2 Generator Module Verification	15
4	Review and Implementation of the Hazard Detection Unit	16
5	Implementation of the Final Stages (Memory and Write Back) and Execution of the Benchmark Program (Bubble Sort)	18
5.1	Data Memory Stage	18
5.2	Write Back Stage	18
6	Implementation and Analysis of the Forwarding Technique	20

1 Simulation Results of the Fetch Stage (IF) and Pipeline Registers

The following figure shows the simulation result of the circuit designed for the IF stage and pipeline registers in the ModelSim software:

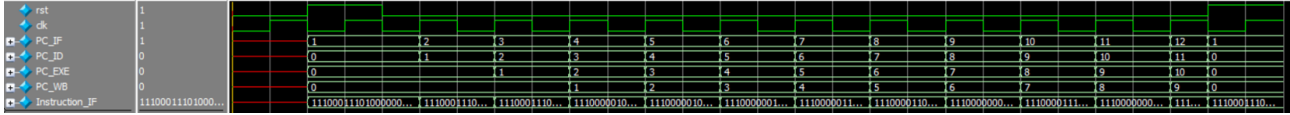


Figure 1: Output waveform of the IF stage simulation

By examining the waveform above, the following points regarding the circuit's operation are confirmed:

- **Correct PC counting:** With each clock pulse, the value of the Program Counter (PC) correctly increments by one, and the counting sequence (0, 1, 2, 3, etc.) is properly maintained.
- **Reset signal operation:** As evident in the signals, whenever the reset signal (rst) is activated, the PC value immediately becomes zero.

Instruction movement in the pipeline: Instructions are correctly read from memory and advance exactly one stage forward in the pipeline registers with each clock pulse.

2 Simulation Results of the Decode Stage (ID Stage)

In this section, the decode module, which includes the Register File, Control Unit, and Condition Check modules, was designed and simulated in the ModelSim software. The purpose of this section is to verify the correct initialization of the registers and to ensure the proper generation of control signals for each instruction. The decode stage is responsible for analyzing the fetched instruction and generating the necessary control signals to guide other parts of the processor (such as the ALU, memory, and register file).

2.1 Condition Check Module

One of the prominent features of the ARM architecture is the Conditional Execution of instructions. All instructions in this processor can be written in such a way that they execute only if a specific condition is met. The Condition Check module is designed precisely to implement this logic.

- **Inputs:** This module receives the 4 condition bits (Cond) from the instruction (bits 28 to 31) and the current status flags of the processor (SR including N, Z, C, V bits).
- **Operation and interaction with the Control Unit:** By matching the condition bits and the status of the flags, this module checks whether the instruction should be executed or not. The output of this module acts as the selector pin for a multiplexer (MUX) placed in the output path of the Control Unit.
- **If the condition is met:** The output of this module allows the control signals generated by the Control Unit to pass through and flow into the pipeline.

- **If the condition is not met:** The output of the Condition Check module sets the multiplexer to a state where all control signals (including write to memory, write to register, etc.) become zero. This action effectively turns the instruction into a No-Operation (NOP) instruction, exiting the pipeline without altering the State of the Machine.

2.2 Register File Initialization Check

According to the manual, this module has 15 registers. To test the correct operation of this section, all registers were initially loaded precisely with their own numbers (for example, the value of register R1 is 1, register R5 is 5, etc.). The image below shows the simulation output and the population of the registers with their respective numbers.

Address	Value
00000000	00000000000000000000000000000000
00000004	00000000000000000000000000000001
00000008	00000000000000000000000000000010
0000000c	00000000000000000000000000000011
00000010	00000000000000000000000000000100
	00000000000000000000000000000101
	00000000000000000000000000000110
	00000000000000000000000000000111
	00000000000000000000000000001000
	00000000000000000000000000001001
	00000000000000000000000000001010
	00000000000000000000000000001011
	00000000000000000000000000001100
	00000000000000000000000000001101
	00000000000000000000000000001110

Figure 2: Initializing the Register File with register numbers

2.3 Control Unit Performance Verification

This module is the core of control signal generation in the processor. Based on the standard design principles of this lab, the detection of the instruction type (Op-code) must be performed exclusively in this module, and the direct transfer of the Op-code to other pipeline stages is not permitted.

- **Inputs:** This module receives the essential instruction fields, including 2 Mode bits (to identify the instruction category, whether computational, memory, or branch), 4 Op-code bits (the operation code), and the S bit (determining the update of the status register).
- **Outputs (Unconditional signals):** Based on its inputs, this unit generates raw (unconditional) control signals, which include the following:
 - **EXE_CMD:** A 4-bit signal that specifies the exact operation type for the Arithmetic Logic Unit (ALU) in the execution stage.
 - **MEM_R_EN and MEM_W_EN:** Signals related to reading and writing in the data memory (corresponding to LDR and STR instructions).
 - **WB_Enable:** The enable signal for the Write Back stage to write the result into the register file.
 - **B:** The signal identifying Branch instructions.
 - **S:** The signal confirming the status register update (which in computational instructions is exactly equivalent to the input S bit).

To check the proper operation of the Control Unit and whether it generates the control signals (such as write to memory, read from memory, register update, etc.) correctly, we loaded the following instructions into the Instruction Memory and examined the outputs:

```

32'b1110_00_0_0000_0_0000_0000_000000000000; // NOP
32'b1110_00_1_1101_0_0000_0001_000000000101; // MOV R1,#5
32'b1110_00_1_1111_0_0000_0010_000000000001; // MVN R2,#1
32'b1110_00_0_0100_0_0001_0011_000000000010; // ADD R3,R1,R2
32'b1110_00_0_0101_0_0001_0100_000000000010; // ADC R4,R1,R2
32'b1110_00_0_0010_0_0001_0101_000000000010; // SUB R5,R1,R2
32'b1110_00_0_0110_0_0001_0110_000000000010; // SBC R6,R1,R2
32'b1110_00_0_0000_0_0001_0111_000000000010; // AND R7,R1,R2
32'b1110_00_0_1100_0_0001_1000_000000000010; // ORR R8,R1,R2
32'b1110_00_0_0001_0_0001_1001_000000000010; // EOR R9,R1,R2
32'b1110_00_0_1010_1_0001_0000_000000000010; // CMP R1,R2
32'b1110_00_0_1000_1_0001_0000_000000000010; // TST R1,R2
32'b1110_01_0_0100_1_0000_1010_000000000000; // LDR R10,[R0]
32'b1110_01_0_0100_0_0000_0001_000000000100; // STR R1,[R0,#4]
32'b1110_10_1_0_000000000000000000000000; // B 0

```

MOV Instruction (Move constant value):

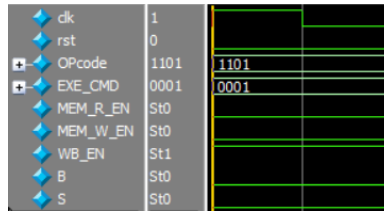


Figure 3: Output of the MOV instruction

- **Signal Analysis:** The goal of this instruction is to write a value into a register (R1). Therefore, the WB_EN signal must become 1. Since there is no need for memory, branching, or changing flags, the MEM_R_EN, MEM_W_EN, B, and S signals will be zero.

MVN Instruction (Move NOT):

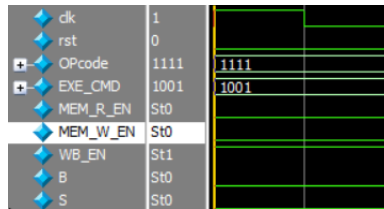
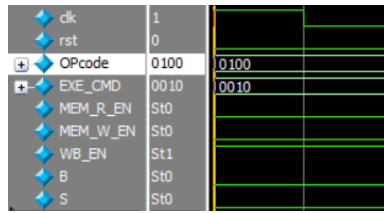


Figure 4: Output of the MVN instruction

- **Signal Analysis:** Similar to the MOV instruction, the result here is also written into a register (R2), so WB_EN is equal to 1. All other control signals (MEM_R_EN, MEM_W_EN, B, and S) are zero.

ADD Instruction (Addition):

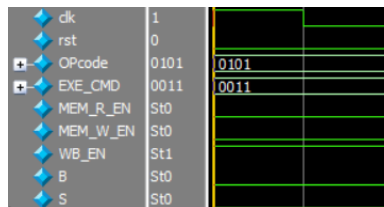


Signal	Value	Output 1	Output 2
clk	1		
rst	0		
Opcode	0100	0100	
EXE_CMD	0010	0010	
MEM_R_EN	St0		
MEM_W_EN	St0		
WB_EN	St1		
B	St0		
S	St0		

Figure 5: Output of the ADD instruction

- **Signal Analysis:** This is a computational instruction where the result must be written into the destination register (R3), consequently WB_EN becomes 1. We do not interact with memory or have a branch, so the rest of the signals are zero. (The EXE_CMD operation code is set accordingly for the addition operation).

ADC Instruction (Add with Carry):

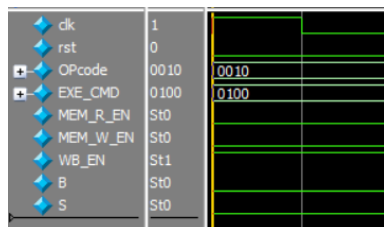


Signal	Value	Output 1	Output 2
clk	1		
rst	0		
Opcode	0101	0101	
EXE_CMD	0011	0011	
MEM_R_EN	St0		
MEM_W_EN	St0		
WB_EN	St1		
B	St0		
S	St0		

Figure 6: Output of the ADC instruction

- **Signal Analysis:** The result of this computational operation is stored in the destination register (R4), therefore WB_EN must be 1. The signals related to memory, branching, and flag updating (MEM_R_EN, MEM_W_EN, B, and S) are zero.

SUB Instruction (Subtraction):



Signal	Value	Output 1	Output 2
clk	1		
rst	0		
Opcode	0010	0010	
EXE_CMD	0100	0100	
MEM_R_EN	St0		
MEM_W_EN	St0		
WB_EN	St1		
B	St0		
S	St0		

Figure 7: Output of the SUB instruction

- **Signal Analysis:** Similar to other computational instructions, the subtraction result must be stored in a register (R5), so the WB_EN signal is equal to 1. The remaining control signals are zero.

SBC Instruction (Subtract with Carry):

clk	1	
rst	0	
Opcode	0110	0110
EXE_CMD	0101	0101
MEM_R_EN	St0	
MEM_W_EN	St0	
WB_EN	St1	
B	St0	
S	St0	

Figure 8: Output of the SBC instruction

- **Signal Analysis:** In this instruction as well, the final value is written into a register (R6), thus WB_EN will be 1. All other control signals (MEM_R_EN, MEM_W_EN, B, and S) are zero.

AND Instruction (Logical AND):

clk	1	
rst	0	
Opcode	0000	0000
EXE_CMD	0110	0110
MEM_R_EN	St0	
MEM_W_EN	St0	
WB_EN	St1	
B	St0	
S	St0	

Figure 9: Output of the AND instruction

- **Signal Analysis:** This is a logical instruction whose result is stored in a register (R7), therefore WB_EN must be 1. There is no need to access memory or branch, so the other signals are zero.

ORR Instruction (Logical OR):

clk	1	
rst	0	
Opcode	1100	1100
EXE_CMD	0111	0111
MEM_R_EN	St0	
MEM_W_EN	St0	
WB_EN	St1	
B	St0	
S	St0	

Figure 10: Output of the ORR instruction

- **Signal Analysis:** The result of the OR operation is written into the destination register (R8), so the WB_EN signal is equal to 1. The rest of the control signals are zero.

EOR Instruction (Logical Exclusive OR):

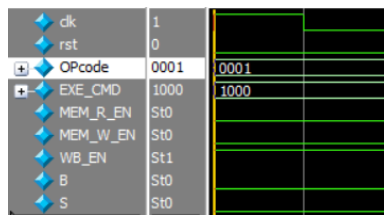


Figure 11: Output of the EOR instruction

- **Signal Analysis:** Like other logical instructions, the result is stored in a register (R9) and WB_EN becomes 1. The signals related to memory and branching are all zero.

CMP Instruction (Compare):

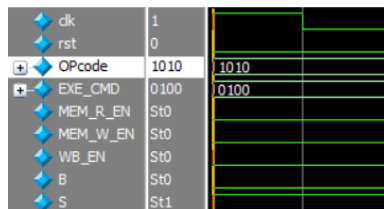


Figure 12: Output of the CMP instruction

- **Signal Analysis:** In the compare instruction, no result is stored in the registers, therefore WB_EN is strictly zero. The sole purpose of this instruction is to update the Status flags, consequently the S signal must be 1. The other signals are zero.

TST Instruction (Test Bits):

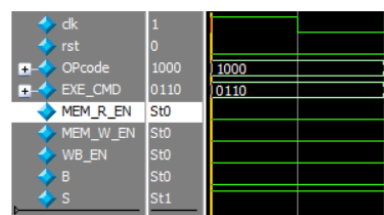


Figure 13: Output of the TST instruction

- **Signal Analysis:** This instruction (performing a logical AND for testing) does not store a value in a register, so WB_EN is zero. However, since the status flags must be updated, the S signal becomes 1. The rest of the signals are zero.

LDR Instruction (Load from Memory):

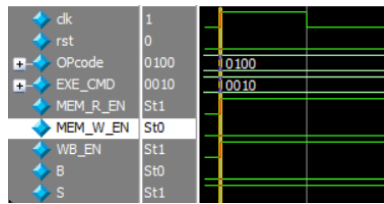


Figure 14: Output of the LDR instruction

- **Signal Analysis:** In this instruction, the goal is to read data from memory and then write that data into a register. Therefore, both the MEM_R_EN and WB_EN signals must become 1. The write-to-memory signal (MEM_W_EN) and the rest are zero.

STR Instruction (Store in Memory):

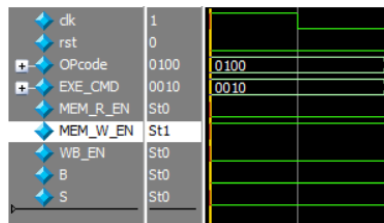


Figure 15: Output of the STR instruction

- **Signal Analysis:** In this instruction, the goal is to write the value of a register into memory. Therefore, the MEM_W_EN signal must become 1. Since nothing is supposed to be written into the register, WB_EN is zero, and MEM_R_EN will also be zero.

B Instruction (Branch):

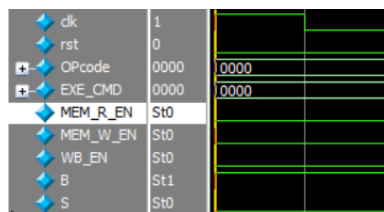


Figure 16: Output of the B instruction

- **Signal Analysis:** In this branch instruction, the only signal that must become 1 is the B (Branch) signal so that the PC value changes. The rest of the signals, including WB_EN, MEM_R_EN, MEM_W_EN, and S, will all be zero.

3 Simulation and Verification of the Execution Stage (EXE Stage)

In this section, we examine and evaluate the Execution Stage (EXE Stage) in the processor. To ensure the correct operation of various modules in this stage (such as the ALU, Status Register, etc.), different test programs were loaded into the Instruction Memory, and the behavior of each circuit was independently simulated in the ModelSim software.

3.1 Arithmetic Logic Unit (ALU) Verification

To check and confirm the operation of the Arithmetic Logic Unit (ALU) in the processor, a set of instructions including arithmetic and logical operations was placed in the Instruction Memory. The loaded machine codes along with their assembly equivalents are as follows:

```
32'b1110_00_0_0100_0_0001_0011_000000000010; // ADD R3, R1, R2
32'b1110_00_0_0010_0_1010_0100_000000000010; // SUB R4, R10, R2
32'b1110_00_0_0000_0_0101_0101_000000000110; // AND R5, R5, R6
32'b1110_00_0_1100_0_0111_0111_000000001000; // ORR R7, R7, R8
32'b1110_00_0_1101_0_0000_1001_000000001100; // MOV R9, R12
32'b1110_00_0_1111_0_0000_1011_000000000001; // MVN R11, R1
32'b1110_00_0_0010_1_0010_1010_000000000001; // SUBS R10, R2, R1
32'b1110_00_0_0101_0_0001_1100_000000000010; // ADC R12, R1, R2
32'b1110_00_0_0110_0_0011_1101_000000000001; // SBC R13, R3, R1
32'b1110_00_0_0001_0_0001_1110_000000000010; // EOR R14, R1, R2
```

After executing these instructions in the ModelSim software, the following waveform was obtained:

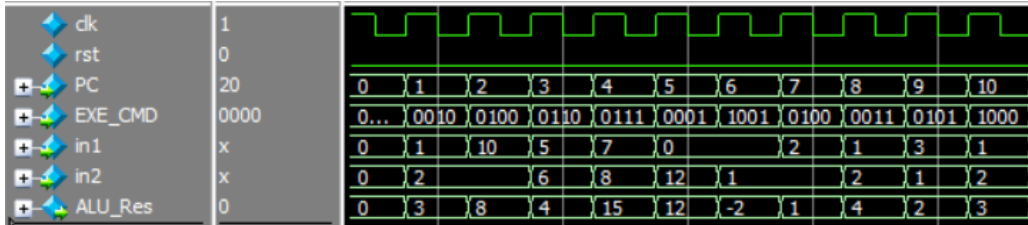


Figure 17: Simulation results of ALU performance for arithmetic and logical instructions

Next, the performance analysis of the ALU for each instruction based on the waveform results is presented:

1. ADD Instruction (Addition Test): ADD R3, R1, R2

- **Goal:** Check the ALU's ability to calculate the sum of two values ($1 + 2$).
- **Expected Output:** The ALU_Res signal should be equal to 3.
- **Machine Code:** 32'b1110_00_0_0100_0_0001_0011_000000000010;
- **Status in Waveform:** As indicated in the waveform, the output shows exactly the value 3, confirming the correctness of this operation.

2. SUB Instruction (Subtraction Test): SUB R4, R10, R2

- **Goal:** Check the ALU's ability to calculate subtraction ($10 - 2$).
- **Expected Output:** The ALU_Res signal should be equal to 8.
- **Machine Code:** 32'b1110_00_0_0010_0_1010_0100_000000000010;
- **Status in Waveform:** The output in the waveform has become 8, proving the correctness of the subtraction operation.

3. AND Instruction (Logical AND Test): AND R5, R5, R6

- **Goal:** Perform a bitwise logical AND operation ($5 \& 6$).
- **Expected Output:** Given the binary values of the numbers, the ALU_Res signal should be equal to 4.
- **Machine Code:** 32'b1110_00_0_0000_0_0101_0101_000000000110;
- **Status in Waveform:** The output displays the value 4, which perfectly matches the manual calculations.

4. ORR Instruction (Logical OR Test): ORR R7, R7, R8

- **Goal:** Perform a bitwise logical OR operation ($7 \mid 8$).
- **Expected Output:** Given the binary values of the numbers, the ALU_Res signal should be equal to 15.
- **Machine Code:** 32'b1110_00_0_1100_0_0111_0111_000000001000;
- **Status in Waveform:** In the waveform, the value 15 has correctly appeared in the output.

5. MOV Instruction (Data Transfer Test): MOV R9, R12

- **Goal:** Check the ability of the ALU data paths to directly transfer the second input to the output.
- **Expected Output:** The value of register 12 must be transferred, thus ALU_Res should be 12.
- **Machine Code:** 32'b1110_00_0_1101_0_0000_1001_000000001100;
- **Status in Waveform:** The ALU output displays the number 12, and the operation is successful.

6. MVN Instruction (NOT Test): MVN R11, R1

- **Goal:** Check the inversion of all bits of the second operand (NOT 1).
- **Expected Output:** The bitwise NOT of the number 1 in a 32-bit two's complement system is equivalent to 0xFFFFFFFF, which is -2.
- **Machine Code:** 32'b1110_00_0_1111_0_0000_1011_000000000001;
- **Status in Waveform:** The output in the waveform is the number -2, which indicates the correctness of this operation.

7. SUBS Instruction (Preparing the Carry Flag): SUBS R10, R2, R1

- **Goal:** Generate the Carry flag (C=1) with a subtraction without Borrow (2 - 1) to test subsequent instructions.
- **Expected Output:** ALU_Res should be 1, and the C flag should turn on.
- **Machine Code:** 32'b1110_00_0_0010_1_0010_1010_000000000001;
- **Status in Waveform:** The output value has become 1.

8. ADC Instruction (Add with Carry Test): ADC R12, R1, R2

- **Goal:** Calculate (1 + 2 + C).
- **Expected Output:** Given that C=1 from the previous instruction, the result should be 4.
- **Machine Code:** 32'b1110_00_0_0101_0_0001_1100_000000000010;
- **Status in Waveform:** The output correctly displays the number 4, confirming the effect of the carry flag.

9. SBC Instruction (Subtract with Carry Test): SBC R13, R3, R1

- **Goal:** Calculate the formula (in1 - in2 - $\sim C$), which is equivalent to subtraction taking Borrow into account.
- **Expected Output:** Register 3 (which became 3 in the first instruction) minus Register 1, and since C=1 ($\sim C=0$), the result (3 - 1 - 0) should be 2.
- **Machine Code:** 32'b1110_00_0_0110_0_0011_1101_000000000001;
- **Status in Waveform:** An output of 2 is obtained, which is completely correct.

10. EOR Instruction (Exclusive OR Test): EOR R14, R1, R2

- **Goal:** Perform a bitwise XOR logical operation ($1 \wedge 2$).
- **Expected Output:** The ALU_Res signal should be 3.
- **Machine Code:** 32'b1110_00_0_0001_0_0001_1110_000000000010;
- **Status in Waveform:** The final clock of the waveform correctly displays the value 3.

3.2 Status Register Verification and Flag Generation

In this section, the goal is to evaluate the correct generation of status flags by the ALU module and their storage in the Status Register. The status_bits signal is a 4-bit output whose bit valuations, according to the ARM architecture, are as follows:

- status_bits[3] : N (Negative - indicates a negative result)
- status_bits[2] : Z (Zero - indicates a zero result)
- status_bits[1] : C (Carry - indicates the presence of a carry or no need to borrow)
- status_bits[0] : V (Overflow - indicates the occurrence of an overflow in signed calculations)

To accurately test these flags, we first loaded specific values into the registers using the first 5 instructions, and then with the next 4 instructions (which all have an S suffix and update the flags) we examined all four states. The contents of the instruction memory along with descriptions are as follows:

```
32'b1110_00_1_1101_0_0000_1000_000000000101; // MOV R8,#5
32'b1110_00_1_1101_0_0000_1001_000000000100; // MOV R9,#4
32'b1110_00_1_1111_0_0000_1011_000000000000; // MVN R11,#0
32'b1110_00_1_1101_0_0000_1100_000000000001; // MOV R12,#1
32'b1110_00_1_1111_0_0000_1101_000100000010; // MVN R13,#0x80000000
32'b1110_00_0_0010_1_1000_1010_000000001000; // SUBS R10,R8,R8 (Z)
32'b1110_00_0_0010_1_1001_1010_000000001000; // SUBS R10,R9,R8 (N)
32'b1110_00_0_0100_1_1011_1010_000000001100; // ADDS R10,R11,R12 (C)
32'b1110_00_0_0100_1_1101_1010_000000001101; // ADDS R10,R13,R13 (V)
```

The simulation output for these instructions in the ModelSim software is as follows:

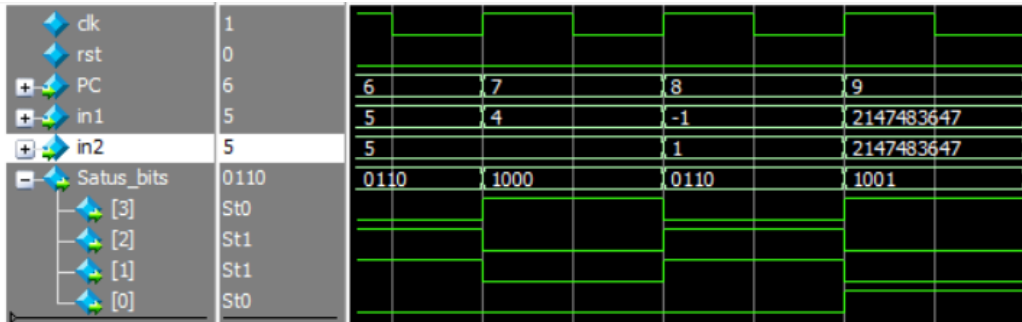


Figure 18: Waveform for verifying the performance of status flags N, Z, C, V

By examining the Satus_bits signal values at different clocks, the correctness of the circuit's operation is confirmed for the following reasons:

- **Testing the Z flag at PC = 6:** At this clock, the SUBS R10, R8, R8 instruction is executed. Both inputs are 5 (in1=5, in2=5). The result of the subtraction is zero, therefore the Z flag (bit 2) becomes one. Also, in the subtraction operation, since there was no need to borrow, the C flag (bit 1) also becomes one. As a result, the Satus_bits value correctly becomes 0110.

- **Testing the N flag at PC = 7:** The SUBS R10, R9, R8 instruction is executed (in1=4, in2=5). The result of subtracting 5 from 4 is -1. Since the result is negative, the N flag (bit 3) becomes one, and since we needed a Borrow, the C flag becomes zero. Therefore, the Satus_bits value correctly shows 1000.
- **Testing the C flag at PC = 8:** The ADDS R10, R11, R12 instruction is executed. The inputs are -1 (0xFFFFFFFF) and 1. Adding these two numbers makes the result zero and generates a Carry. As is evident, both the C flag (bit 1) and the Z flag (bit 2) have become one, and the Satus_bits output is 0110, which is completely correct.
- **Testing the V flag at PC = 9:** The ADDS R10, R13, R13 instruction is executing. Two 2147483647 numbers (the largest positive number in a 32-bit system) are added together. The result of this addition exceeds the capacity for positive numbers and enters the negative range (overflow). The waveform shows that the V flag (bit 0) has become one due to the occurrence of overflow, and the N flag (bit 3) has become one due to the negative result, proving the complete correctness of this design with a value of 1001 for Satus_bits.

3.3 Branch Address Calculation Verification

In this section, the goal is to evaluate the circuit's performance in correctly calculating the destination address for Branch instructions in the Execution Stage (EXE Stage). According to the manual, to calculate the branch address, the 24-bit values (Immediate) are first Sign Extended and then directly added (without left shifting) to the PC value.

To test this circuit, a branch instruction (B 8) was placed among other instructions in the memory. The loaded machine codes along with their equivalents are as follows:

```

32'b1110_00_0_0100_0_0001_0011_000000000010; //ADD R3, R1, R2
32'b1110_10_1_0_0000000000000000000000001000; //B 8 (Branch to address PC + 8)
32'b1110_00_0_0100_0_0001_0011_000000000010; //ADD R3, R1, R2
32'b1110_00_0_0010_0_1010_0100_000000000011; //SUB R4, R10, R3
32'b1110_00_0_0000_0_0101_0101_000000000110; //AND R5, R5, R6
32'b1110_00_0_1100_0_0111_0111_000000001000; //ORR R7, R7, R8
32'b1110_00_0_1101_0_0000_1001_000000001100; //MOV R9, R12
32'b1110_00_0_0100_0_0001_0011_000000000010; //ADD R3, R1, R2
32'b1110_00_0_0010_0_1010_0100_000000000011; //SUB R4, R10, R3
32'b1110_00_0_0000_0_0101_0101_000000000110; //AND R5, R5, R6
32'b1110_00_0_1100_0_0111_0111_000000001000; //ORR R7, R7, R8 (This is our branch destination)
32'b1110_00_0_1101_0_0000_1001_000000001100; //MOV R9, R12

```

The output waveform for this test in the ModelSim software is as follows:

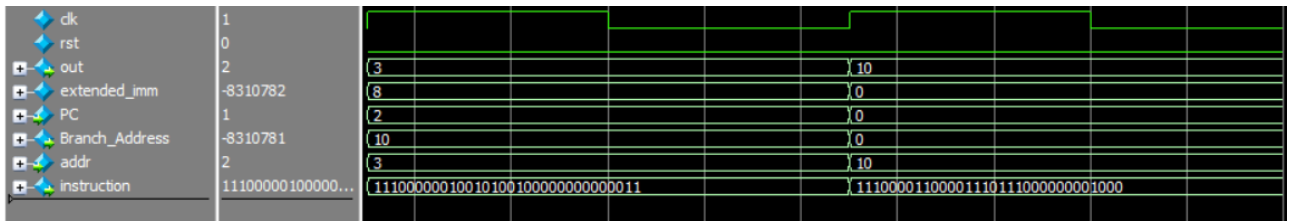


Figure 19: Waveform for verifying branch address calculation and instruction transfer

Analysis of Simulation Results:

According to the waveform, the execution of the B 8 instruction can be examined precisely:

- **Calculation of the branch value:** During the execution of the branch instruction in the ALU unit, the extended_imm value (the extended value of the branch instruction) is correctly equal to 8.
- **Calculation of the destination address (Branch_Address):** The value of the PC register, which is concurrently in the execution stage alongside this instruction, is 2. By adding these two values ($2 + 8$), the Branch_Address output is correctly calculated as 10.
- **Checking for a successful branch:** In the next clock, it is observed that the addr signal (which represents the fetch address or PC) directly jumps from 3 to 10.
- **Verification of correct destination instruction fetch:** Simultaneously with the address changing to 10, the instruction signal changes to the value 111000001100001110111000000001000. This value is exactly equivalent to the binary code of instruction number 10 (ORR R7, R7, R8) in the instruction memory, proving that the branch circuit operated successfully and entirely correctly.

3.4 Val2 Generator Module Verification

In this section, the goal is to evaluate the functional correctness of the Val2 Generator module when encountering various types of second operands. This module is considered one of the key processing components in the Execution Stage, tasked with receiving the trailing twelve bits of the instruction (shifter_operand) alongside the value read from the register (Val_Rm) and, after necessary processing, preparing the final 32-bit operand to feed the Arithmetic Logic Unit (ALU).

Given the architecture of the ARM processor designed in this experiment, the Val2 Generator module must be able to correctly identify and implement four different structures:

- **Immediate with Rotate:** In data processing instructions, when the I bit is one, this module reads an 8-bit data (immed_8) and places it into a 32-bit container. It then rotates this value to the right (Rotate Right) by twice the 4-bit rotation value (rotate_imm) to construct the final 32-bit number.
- **Register Direct:** If the second operand is directly a register and no shifting is applied to it (such as a simple ADD instruction), the module simply passes the 32-bit value read from the Rm register directly and unchanged to the output.
- **Immediate Shift on Register:** In the case where the I bit is zero but the instruction includes a shift operation, the module receives the value read from the Rm register. Then, based on the 5 bits determining the shift amount (shift_imm) and the 2 bits determining the shift type (LSL, LSR, ASR, or ROR), it implements the bit shifting operation on it.
- **Memory Offset:** For load and store instructions in the data memory (LDR and STR instructions), the second input to the ALU must equal the 12-bit extended offset. Under these conditions, the module extracts the trailing 12 bits of the instruction and, by extending it to 32 bits, sends the offset value to the ALU to calculate the final address.

```
11100011101000000001010011111111; // MOV R1, #0xFF000000
11100000100000010011000000000010; // ADD R3, R1,
11100001101000000100000110000010; // MOV R4, R2, LSL #
11100100100100010101000000001100; // LDR R5, [R1, #12]
```

The simulation output for these instructions in the simulator software is as follows:

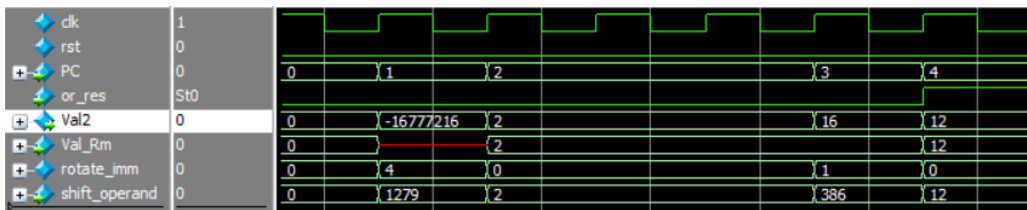


Figure 20: Waveform verifying the performance of the Val2 Generator module

- **Testing the Immediate state in PC = 1:**

At this clock, the MOV R1, #0xFF000000 instruction is executed. The shift_operand signal displays the value 1279 (binary equivalent 0100_1111_1111), where the 8 data bits equal 255 (or FF in hex) and the 4 rotate bits (rotate_imm) equal 4.

According to the architecture, this data must be rotated right by twice the rotation amount (i.e., 8 times). The result of this rotation in a 32-bit format becomes 32'hFF000000. Since the simulator display is set to signed decimal, the Val2 signal output correctly displays the value -16777216, which is entirely correct.

- **Testing the Register Direct state in PC = 2:**

At this clock, the ADD R3, R1, R2 instruction is executed. In this state, the I bit is 0 and the value of the second read register (Val_Rm) is 2.

Since no shift is defined for this register, the module must pass the register value unchanged. As evident in the waveform, the Val2 output is exactly 2, confirming the correctness of this path's operation.

- **Testing the Immediate Shift on Register state in PC = 3:**

At this clock, the MOV R4, R2, LSL #3 instruction is executed. The input register value (Val_Rm) is 2, and the shift_operand signal shows the value 386 (binary equivalent 0001_1000_0010).

The 5 bits corresponding to the shift amount in this instruction equal the number 3, and the shift type equals 00 (LSL or logical shift left). By applying a 3-bit left shift on the number 2, the input value is multiplied by 8 and should become 16.

The Val2 signal output correctly shows the number 16.

- **Testing the Memory Offset (LDR) state in PC = 4:**

At this clock, the LDR R5, [R1, #12] instruction is executing. For load and store instructions in memory (LDR/STR), the module must extend the 12-bit offset at the end of the instruction and send it directly as the second operand.

At this clock, the input offset value is 12, and the Val2 output signal also shows exactly the value 12, indicating the module's correct operation in load and store mode.

4 Review and Implementation of the Hazard Detection Unit

In Pipelined processors, the simultaneous execution of instructions can lead to various Hazards. Given that structural hazards have been resolved by adjusting the timing of reading and writing to the register file, the main task of the Hazard Detection Unit is to identify and resolve Read After Write (RAW) data hazards.

The RAW hazard occurs when the current instruction in the decode (ID) stage requires data that was calculated by a previous instruction but has not yet been written back into the register file. To detect this condition, this module compares the source register addresses of the current instruction (src1 and, if needed, src2) with the destination register addresses of the instructions currently in the execution (EXE) and memory (MEM) stages. If there is a match and the write to register signal (WB_EN) is active for those stages, the Hazard_Detected signal becomes one.

As soon as a hazard is detected, to prevent faulty execution, this module takes three simultaneous actions to Stall the pipeline:

- **PC Stall:** Activating the program counter freeze signal so that no new instruction is fetched.
- **IF/ID Stall:** Activating the freeze signal of the intermediate register to hold the current instruction in the decode stage.
- **Bubble Injection:** Zeroing out all control signals exiting the ID stage (turning the instruction into a NOP) to prevent invalid data from entering the EXE stage.

To evaluate the performance of this module in the face of sequential Data Dependencies, a set of instructions was placed in memory whose execution depends on the results of the instructions immediately preceding them. The binary codes along with their assembly equivalents are as follows:

```

1110001110100000000010000000000101; // MOV R1, #5
11100011101000000000100000000001010; // MOV R2, #10
1110000010000000100110000000000010; // ADD R3, R1, R2 (Dependent on R1 and R2)
111000000100001101000000000000001; // ADD R4, R3, R1 (Dependent on R3)
1110000000000001101010000000000100; // AND R5, R3, R4 (Dependent on R3 and R4)
1110000101010011000000000000000100; // CMP R5, R4 (Dependent on R5 and R4)
111001001000010001010000000000000; // LDR R5, [R4, #0] (Dependent on R4)
111001001001010001100000000000000; // LDR R6, [R4, #0]
111000011000000101100000000000010; // ORR R6, R1, R2
11100000000000000000000000000000; // NOP

```

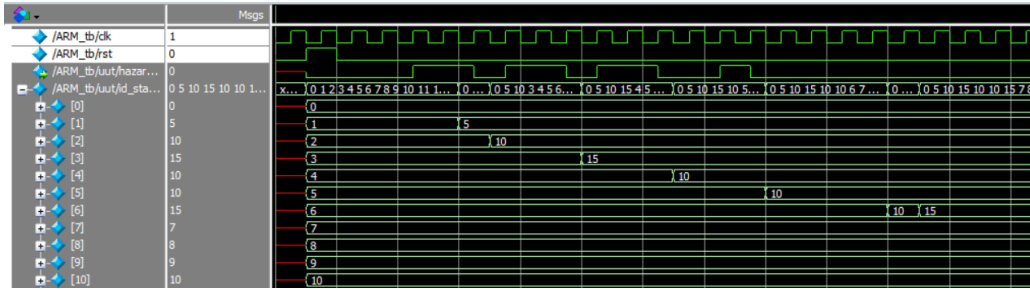


Figure 21: Waveform resulting from testing the hazard module

By observing the obtained waveform, the flawless operation of the hazard detection unit is proven. The execution process is as follows:

At the beginning of execution, the values 5 and 10 are supposed to be written into registers R1 and R2, respectively. When the third instruction (ADD R3, R1, R2) enters the decode stage, it requires the values of these two registers. However, since the MOV instructions are still in the subsequent pipeline stages (EXE or MEM), the hazard detection unit identifies this RAW dependency and sets the hazard signal (the wave connected to the module) to High (1). By asserting this signal, it can be observed that the PC register value remains constant (is not updated) for a few clock cycles, and no new instructions are fetched. Simultaneously, the control signals headed to the execution stages are zeroed, and a bubble (NOP) is injected into the pipeline to provide enough time for the values of R1 and R2 to be written back into the register file.

After the data 5 and 10 are successfully written into the register file, the hazard signal goes to zero, and the pipeline resumes operation. As evident in the register file signals, the ADD

instruction executes with the correct data, and register [3] correctly receives the value 15 (the sum of 5 and 10).

This stalling process is also successfully repeated for subsequent instructions that have chained dependencies (such as the ADD R4, R3, R1 instruction waiting for the value 15 in R3), and ultimately, the computational values for registers [4], [5], and [6] are recorded in the registers without any data interference or error.

5 Implementation of the Final Stages (Memory and Write Back) and Execution of the Benchmark Program (Bubble Sort)

After ensuring the correct operation of the pipeline when encountering hazards, the two final processor stages, namely the memory stage (MEM) and write back (WB), were designed and connected according to the pipeline architecture.

5.1 Data Memory Stage

In this stage, the processor interacts with the data memory. The data memory is described in the code as an Array of 32-bit registers, which allows read and write access concurrently with the pipeline.

- **Addressing:** The access address to this memory is supplied directly from the output calculated in the ALU unit (ALU_res).
- **Writing to memory (STR):** If the MEM_W_EN control signal (which has been routed from the decode stage to this stage in the pipeline) is active, the data present in the source register (Val_Rm or Dest) is written to the calculated address.
- **Reading from memory (LDR):** The output of this memory always reflects the data present at the ALU_res address, which will be used in the next stage if the instruction requires it.

5.2 Write Back Stage

This stage, acting as the final part of the pipeline, is responsible for selecting the final data to be recorded in the register file. The logic of this section is implemented using a 32-bit multiplexer (MUX):

- The selector pin of this multiplexer is connected to the MEM_R_EN control signal.
- If the instruction is a load from memory (MEM_R_EN = 1), the data read from the Data Memory is routed towards the register file.
- For other computational and logical instructions (MEM_R_EN = 0), the calculation result of the arithmetic logic unit (ALU_res), which was stored in the intermediate pipeline register, is selected for write back.

By transferring this output to the register file (provided the WB_EN signal is active), the instruction execution cycle in the ARM processor is completed.

To evaluate the integrated performance of the processor (including the correctness of ALU operation, conditional branches, memory access, and especially the hazard detection unit's performance in a real scenario), the benchmark program provided in the manual, which implements the Bubble Sort algorithm, was loaded into the Instruction Memory.

In this algorithm, the processor first stores an array of unsorted data (including positive, negative, and very large numbers) in the memory. Then, through nested loops (using BLT conditional branch and CMP comparison instructions), it reads data from memory, compares them, and swaps them if necessary. At the end of the program, the sorted data is read again and loaded into registers R1 through R6.

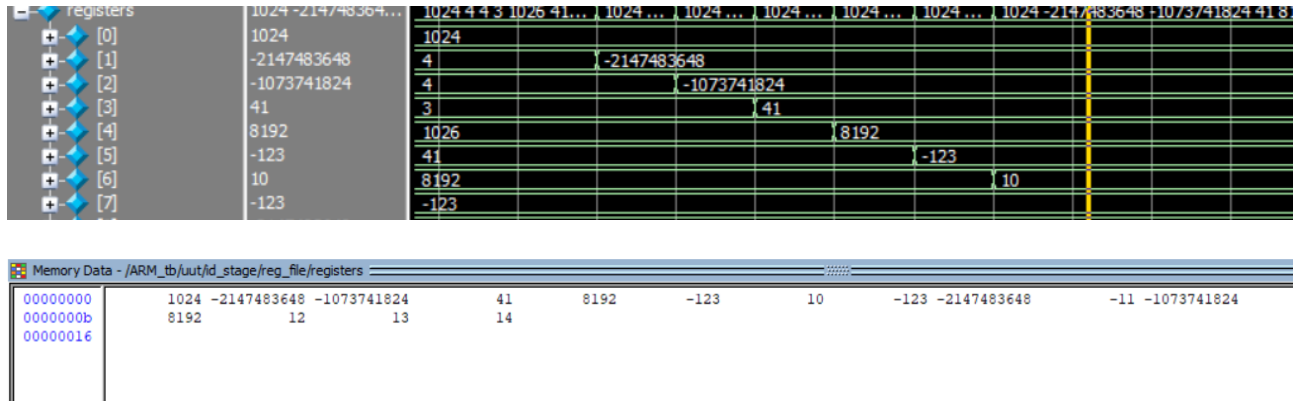


Figure 22: Simulation status and register file after executing Bubble Sort

The register statuses in Signed Decimal format are as follows:

- [R0] = 1024 (As the data memory base address for LDR/STR instructions)
- [R1] = -2147483648
- [R2] = -1073741824
- [R3] = 41
- [R4] = 8192
- [R5] = -123
- [R6] = 10

As specified in the lab manual, the criterion for the correct execution of this algorithm is the ascending sort of data in registers R1 to R4. By examining the values above, it is clearly evident that the ascending condition $(-2147483648 < -1073741824 < 41 < 8192)$ is perfectly met.

6 Implementation and Analysis of the Forwarding Technique

In the Pipeline architecture implemented previously, Read After Write (RAW) data hazards were managed exclusively through stalling the pipeline (Stall) and injecting bubbles (Bubble). These stalls, although maintaining the logical correctness of the program, severely reduce the throughput and efficiency of the processor.

The Forwarding technique is designed to overcome this limitation. The foundation of this technique rests on the principle that the data calculated in the execution stage (EXE) or the data ready in the memory stage (MEM), before being written back into the registers, is sent directly to the input of the calculation unit (ALU) for the next instruction via Bypass Paths. It should be noted that not all data hazards can be resolved with Forwarding. A Load hazard (dependency on data being read from memory by an LDR instruction) cannot be resolved instantaneously with Forwarding; because the data is not ready until the end of the MEM stage. Therefore, in this specific case, the processor still requires the injection of a Stall cycle. To add the Forwarding capability, two 3-to-1 multiplexers were embedded in the ALU input paths (after the intermediate ID/EXE register) to select the correct data from the following sources:

- **Register file data:** Normal state (without hazard).
- **Memory stage data (Forward from MEM):** The ALU output related to the immediately preceding instruction.
- **Write back stage data (Forward from WB):** The output related to the instruction two cycles prior.

Additionally, a Forwarding_Unit module was designed which generates the selector signals (sel_src1, sel_src2) by comparing the source register addresses (src1, src2) with the destination register addresses in the EXE and MEM stages.

To measure performance, the benchmark program (Bubble Sort) was executed in two modes: enabled (Forwarding_EN = 1) and disabled (Forwarding_EN = 0). The results obtained from the ModelSim environment are as follows:

Simulation output (Number of execution clocks):

- Program execution time without Forwarding: 304 clock cycles
- Program execution time with Forwarding: 195 clock cycles
- Amount of improvement (Clocks saved): 109 clock cycles

```
# -----  
# Execution WITHOUT Forwarding took:          304 clock cycles  
# Execution WITH Forwarding took:             195 clock cycles  
# -----  
# Optimization Results:  
# Clocks Saved (Hazards Prevented):           109 cycles  
# -----
```

Figure 23: Improvement in the number of clock cycles with Forwarding

As evident from the obtained numbers, activating the Forwarding unit drastically reduced the program's execution time. In the disabled state, the processor was forced to stall the pipeline and inject bubbles to resolve computational data dependencies (RAW), wasting many clocks. In the active state, the Forwarding unit, by creating Bypass Paths, sent the required data directly from the EXE and MEM stages to the ALU input, eliminating the need for unnecessary stalls in most instructions. The reduction of 109 clock cycles clearly proves an increase in Throughput and an improvement in processor efficiency. These results confirm that the Forwarding technique was implemented correctly and, alongside the modified Hazard_Detection module (which restricted stalling only to mandatory Load-Use cases), provided optimal performance.

Notes:

First, it is impossible to have a hazard with the instruction three steps back; meaning we either have a hazard with the immediately preceding instruction or the one before that. The reason for this is that if an instruction has a data hazard (we have no control hazards, and structural hazards are resolved for the following reason (since we have one register file module and use it for two instructions during simultaneous reading and writing)) with the instruction three steps back, it means an instruction is in ID and its three preceding instructions are in WB. Now, since we write to our register file on the falling edge of the clock (meaning in the middle of the clock cycle) and our reading is asynchronous, the updated data is written, the updated data is read, and the ID stage instruction enters Exe with fresh data.

Secondly, when a hazard occurs, it can be handled in two ways:

Only Hazard Unit: The ID instruction might have a hazard with the Exe and Mem instructions, and we should not proceed with stale data in the ID instruction. To do this, if we detect a hazard exists (meaning the previous instructions want to write and their destination is identical to source one or source two (we don't necessarily always have it, which is why we used two_src for instructions that have a second source)), we must stall the pipeline. This means we prevent a new instruction from entering and the IF and ID stage instructions from advancing until the instruction with the hazard reaches the WB stage. At that time, according to the previous note, having two instructions simultaneously in ID and WB presents no hazard and the issue is resolved, after which the pipeline resumes its normal process. (Note that a hazard with the immediately preceding instruction takes precedence over the one two steps back because it is newer, meaning it matters when we have a hazard with both the preceding instruction and the one before that, and it should be placed higher in the hazard unit conditions; although it doesn't matter here, it is important in the case below to be mindful of it). This mode works perfectly, but we are pointlessly stalling the pipeline and wasting clocks because the data is ready, it just hasn't been written to the register file yet. We can cheat and take the data from further ahead and send it further back so that if a hazard is seen, it uses that data. This is precisely the idea of the forwarding unit.

Using the Forwarding Unit alongside the Hazard Unit: For non-load instructions in ID: We let the pipeline advance one stage, meaning now Exe might have a hazard with Mem and WB. As above, we check if a hazard exists (keeping in mind that Mem takes precedence over WB), and if it does, depending on which stage it is, we feed that stage's data to the beginning of Exe instead of its new sources.

For a load instruction in ID: Since we took the wire from the data unit's input, this is actually the address and not the memory data. So for a load, we issue one stall, demonstrating the application of the hazard unit as well (note that for this instruction, instead of stopping for two clocks, we gave one, so forwarding was still useful for us).

In the final section of this report, screenshots of the codes related to the important and main implemented modules have been included.

```

1  module alu (
2      input [3:0] EXE_CMD,
3      input [31:0] in1, in2,
4      input c_in,
5      output [3:0] Satus_bits,
6      output [31:0] out
7  );
8
9      reg [31:0] alu_res;
10     reg n_flag, z_flag, c_flag, v_flag;
11
12     always @(*) begin
13
14         alu_res = 32'd0;
15         n_flag = 1'b0;
16         z_flag = 1'b0;
17         c_flag = 1'b0;
18         v_flag = 1'b0;
19
20         case (EXE_CMD)
21
22             4'b0001: begin
23                 alu_res = in2;
24                 n_flag = alu_res[31];
25                 z_flag = (alu_res == 32'd0);
26             end
27
28             4'b1001: begin
29                 alu_res = ~in2;
30                 n_flag = alu_res[31];
31                 z_flag = (alu_res == 32'd0);
32             end
33
34             4'b0010: begin
35                 {c_flag, alu_res} = in1 + in2;
36                 n_flag = alu_res[31];
37                 z_flag = (alu_res == 32'd0);
38                 v_flag = (in1[31] == in2[31]) && (alu_res[31] != in1[31]);
39             end
40
41             4'b0011: begin
42                 {c_flag, alu_res} = in1 + in2 + c_in;
43                 n_flag = alu_res[31];
44                 z_flag = (alu_res == 32'd0);
45                 v_flag = (in1[31] == in2[31]) && (alu_res[31] != in1[31]);
46             end
47
48             4'b0100: begin
49                 alu_res = in1 - in2;
50                 n_flag = alu_res[31];
51                 z_flag = (alu_res == 32'd0);
52                 c_flag = (in1 >= in2);
53                 v_flag = (in1[31] != in2[31]) && (alu_res[31] == in2[31]);
54             end
55
56             4'b0101: begin
57                 {c_flag, alu_res} = (c_in) ? (in1 - in2) : (in1 - in2 - 32'd1) ;
58                 n_flag = alu_res[31];
59                 z_flag = (alu_res == 32'd0);
60                 v_flag = (in1[31] != in2[31]) && (alu_res[31] == in2[31]);
61             end
62
63             4'b0110: begin
64                 alu_res = in1 & in2;
65                 n_flag = alu_res[31];
66                 z_flag = (alu_res == 32'd0);
67             end
68
69             4'b0111: begin
70                 alu_res = in1 | in2;
71                 n_flag = alu_res[31];
72                 z_flag = (alu_res == 32'd0);
73             end
74
75             4'b1000: begin
76                 alu_res = in1 ^ in2;
77                 n_flag = alu_res[31];
78                 z_flag = (alu_res == 32'd0);
79             end
80
81             default: begin
82                 alu_res = 32'd0;
83                 n_flag = 1'b0;
84                 z_flag = 1'b1;
85                 c_flag = 1'b0;
86                 v_flag = 1'b0;
87             end
88         endcase
89     end
90
91     assign out = alu_res;
92     assign Satus_bits = {n_flag, z_flag, c_flag, v_flag};
93
94 endmodule

```



```

1  module condition_check
2  (
3      input  [3:0] Cond,
4      input  [3:0] SR,
5      output reg out
6  );
7
8      wire N = SR[3];
9      wire Z = SR[2];
10     wire C = SR[1];
11     wire V = SR[0];
12
13     always @(*) begin
14         case (Cond)
15
16             4'b0000: out = Z;
17             4'b0001: out = ~Z;
18
19             4'b0010: out = C;
20             4'b0011: out = ~C;
21
22             4'b0100: out = N;
23             4'b0101: out = ~N;
24
25             4'b0110: out = V;
26             4'b0111: out = ~V;
27
28             4'b1000: out = C && ~Z;
29             4'b1001: out = ~C && Z;
30
31             4'b1010: out = (N==V);
32             4'b1011: out = (N!=V);
33
34             4'b1100: out = ~Z && (N==V);
35             4'b1101: out = Z || (N!=V);
36
37             4'b1110: out = 1'b1;
38             4'b1111: out = 1'b0;
39
40             default: out = 1'b0;
41         endcase
42     end
43
44 endmodule
45

```

```

1  module control_unit (
2      input [3:0] OPCode,
3      input [1:0] Mode,
4      input S,
5      output [8:0] out
6  );
7
8      reg WB_EN, MEM_R_EN, MEM_W_EN, B;
9      reg [3:0] EXE_COM;
10
11     always @(*) begin
12
13         WB_EN = 1'b0;
14         MEM_R_EN = 1'b0;
15         MEM_W_EN = 1'b0;
16         B = 1'b0;
17         EXE_COM = 4'b0000;
18
19         if (Mode == 2'b10) begin
20             B = 1;
21             WB_EN = 0;
22             MEM_R_EN = 0;
23             MEM_W_EN = 0;
24             EXE_COM = 4'b0000;
25         end
26
27         else begin
28             case (OPCode)
29                 4'b0000: begin // AND / NOP
30                     WB_EN = 1;
31                     EXE_COM = 4'b0110;
32                 end
33
34                 4'b0001: begin // EOR
35                     WB_EN = 1;
36                     EXE_COM = 4'b1000;
37                 end
38
39                 4'b0010: begin // SUB
40                     WB_EN = 1;
41                     EXE_COM = 4'b0100;
42                 end
43
44                 4'b0100: begin // ADD / LDR / STR
45                     if (Mode == 2'b00) begin
46                         WB_EN = 1;
47                         EXE_COM = 4'b0010;
48                     end else if (Mode == 2'b01) begin
49                         EXE_COM = 4'b0010;
50                         if (S) begin
51                             MEM_R_EN = 1;
52                             WB_EN = 1;
53                         end
54                     end else begin
55                         MEM_W_EN = 1;
56                     end
57                 end
58
59                 4'b0101: begin // ADC
60                     WB_EN = 1;
61                     EXE_COM = 4'b0011;
62                 end
63
64                 4'b0110: begin // SBC
65                     WB_EN = 1;
66                     EXE_COM = 4'b0101;
67                 end
68
69                 4'b1100: begin // ORR
70                     WB_EN = 1;
71                     EXE_COM = 4'b0111;
72                 end
73
74                 4'b1101: begin // MOV
75                     WB_EN = 1;
76                     EXE_COM = 4'b0001;
77                 end
78
79                 4'b1111: begin // MVN
80                     WB_EN = 1;
81                     EXE_COM = 4'b1001;
82                 end
83
84                 4'b1000: begin // TST
85                     EXE_COM = 4'b0110;
86                 end
87
88                 4'b1010: begin // CMP
89                     EXE_COM = 4'b0100;
90                 end
91
92                 default: begin
93                     EXE_COM = 0;
94                 end
95             endcase
96         end
97     end
98
99     assign out = {WB_EN, MEM_R_EN, MEM_W_EN, EXE_COM, S, B};
100
101 endmodule
102
103
104

```

```

1  module forwarding_unit (
2      input [3:0] src1,
3      input [3:0] src2,
4      input [3:0] MEM_Dest,
5      input [3:0] WB_Dest,
6      input MEM_WB_EN,
7      input WB_WB_EN,
8      output reg [1:0] sel_src1,
9      output reg [1:0] sel_src2
10 );
11
12     always @(*) begin
13         sel_src1 = 2'b00;
14
15         if (MEM_WB_EN && (MEM_Dest == src1)) begin
16             sel_src1 = 2'b01;
17         end
18         else if (WB_WB_EN && (WB_Dest == src1)) begin
19             sel_src1 = 2'b10;
20         end
21     end
22
23     always @(*) begin
24         sel_src2 = 2'b00;
25
26         if (MEM_WB_EN && (MEM_Dest == src2)) begin
27             sel_src2 = 2'b01;
28         end
29         else if (WB_WB_EN && (WB_Dest == src2)) begin
30             sel_src2 = 2'b10;
31         end
32     end
33
34 endmodule

```

```

1  module hazard_detection
2  (
3      input [3:0] MEM_Dest,
4      input MEM_WB_EN,
5      input [3:0] EXE_Dest,
6      input EXE_WB_EN,
7      input [3:0] src1,
8      input [3:0] src2,
9      input Two_src,
10     input forwarding_en,
11     input EXE_MEM_R_EN,
12     output reg hazard
13 );
14
15 always @(*)begin
16
17     if(forwarding_en == 1'b0)
18     begin
19         if((src1 == EXE_Dest) && (EXE_WB_EN))
20             hazard = 1'b1;
21         else if((src2 == EXE_Dest) && (EXE_WB_EN) && (Two_src))
22             hazard = 1'b1;
23         else if((src1 == MEM_Dest) && (MEM_WB_EN))
24             hazard = 1'b1;
25         else if((src2 == MEM_Dest) && (MEM_WB_EN) && (Two_src))
26             hazard = 1'b1;
27         else
28             hazard = 1'b0;
29     end
30
31     else
32     begin
33         if((src1 == EXE_Dest) && (EXE_WB_EN) && (EXE_MEM_R_EN))
34             hazard = 1'b1;
35         else if((src2 == EXE_Dest) && (EXE_WB_EN) && (Two_src) && (EXE_MEM_R_EN))
36             hazard = 1'b1;
37         else
38             hazard = 1'b0;
39     end
40 end
41
42
43 endmodule

```

```

1  module Val2Genrate (
2      input [31:0] Val_Rm,
3      input [11:0] shift_operand,
4      input imm, or_res,
5      output reg [31:0] Val2
6  );
7
8      wire [3:0] rotate_imm = shift_operand[11:8];
9      wire [7:0] immed_8 = shift_operand[7:0];
10     wire [4:0] shift_imm = shift_operand[11:7];
11     wire [1:0] shift_type = shift_operand[6:5];
12
13     wire [31:0] base_val = {24'b0, immed_8};
14     wire [4:0] rotate_amount = rotate_imm * 2;
15
16     always @(*) begin
17         if (or_res) begin
18
19             Val2 = {{20{shift_operand[11]}}, shift_operand};
20         end else begin
21             if (imm) begin
22                 Val2 = (base_val >> rotate_amount) | (base_val << (32 - rotate_amount));
23             end else begin
24                 case (shift_type)
25                     // Logical Shift Left
26                     2'b00: Val2 = Val_Rm << shift_imm;
27
28                     // Logical Shift Right
29                     2'b01: Val2 = Val_Rm >> shift_imm;
30
31                     // Arithmetic Shift Right
32                     2'b10: Val2 = $signed(Val_Rm) >>> shift_imm;
33
34                     // Rotate Right
35                     2'b11: Val2 = (Val_Rm >> shift_imm) | (Val_Rm << (32 - shift_imm));
36
37                     default: Val2 = Val_Rm;
38                 endcase
39             end
40         end
41     end
42
43 endmodule
44

```